into the pit (–).

Figure 3.2. The Q function could be any function that accepts a state and action and returns the value (expected rewards) of taking that action given that state.

Figure 3.3. Schematic of Q-learning with Gridworld. The Q function accepts a state and an action, and returns the predicted reward (value) of that state-action pair. After taking the action, we observe the reward, and using the update formula, we use this observation to update the Q function so it makes better predictions.

Figure 3.4. An illustration of action trajectories leading to the same total reward but that may be valued differently since more recent rewards are generally valued more than distant rewards.

Figure 3.5. The original Q function accepts a state-action pair and returns the value of that state-action pair—a single number. DeepMind used a modified vector-valued Q function that accepts a state and returns a vector of state-action values, one for each possible action given the input state. The vector-valued Q function is more efficient, since you only need to compute the function once for all the actions.

Figure 3.6. In an epsilon-greedy action selection method, we set the epsilon parameter to some value, e.g., 0.1, and with that probability we will randomly select an action (completely ignoring the predicted Q values) or with probability 1 – epsilon = 0.9, we will select the action associated with the highest predicted Q value. An additional helpful technique is to start with a high epsilon value, such as 1, and then slowly decrement it over the training iterations.

Figure 3.7. This is how the Gridworld board is represented as a numpy array. It is a 4 x 4 x 4 tensor, composed of 4 “slices” of a 4 x 4 grid. Each grid slice represents the position of an individual object on the board and contains a single 1, with all other elements being 0s. The position of the 1 indicates the position of that slice’s object.

Figure 3.8. The neural network model we will use to play Gridworld. The model has an input layer that can accept a 64-length game state vector, some hidden layers (we use one, but two are depicted for generality), and an output layer that produces a 4-length vector of Q values for each action, given the state.